UNIVERSITÉ DE SHERBROOKE
Faculté de génie
Département de génie électrique et génie informatique

LOG BOOK

Projet S5

Présenté à
Domingo Palao Munoz

Présenté par
Équipe 1

Sherbrooke – 23 septembre 2026

Table des matières

1 Architecture du système embarqué 1
1.1 Présentation 1
1.2 Communication 1
1.3 Signatures 1
2 Système Linux embarqué 3
2.1 Présentation 3
2.2 PID 1 3
2.3 Paquets ajoutés 3
2.4 Signatures 3
3 Avancements et modification 5
3.1 Avancements 5
3.2 Modification 5
3.3 Signatures 5
4 Formules GoDot 6
4.1 Présentation 6
4.2 Signatures 6
5 Avancements et modification 7
5.1 Avancements 7
5.2 Simulation GoDot (Raphaël Bonhomme) 7
5.3 Modification 7
5.4 Signatures 7
6 Avancements 8
6.1 Capteurs de distance (Jonathan Valade) 8
6.2 Shared memory Region (Marc-Antoine Gauthier) 8
6.3 Fichiers de configuration 8
6.4 Signatures 8

Liste des tableaux

Table des figures

1 Architecture du système embarqué

Date de décision : 15 octobre 2026
Proposé par : Marc-Antoine
Statut : Adopté à l'unanimité
Absences : Aucune

1.1 Présentation

Afin de répondre au besoin du projet, une architecture proposant l'utilisation de plusieurs services représentant une composante unitaire du robot serait souhaitable. De ce fait, l'architecture serait basée sur 4 services principaux.

FSM Permet de prendre des décisions sur l'exécution autonome du robot en fonction des données entrantes.

Sensors Permet l'acquisition des données sensoriels du robots

Motors Envoie les commandes aux moteurs propulseurs ainsi qu'au servo moteur pour la direction

Telemetry Récupère les informations du robot et envoient une copie à un client externe via *TCP*. Permet aussi la réception des commandes.

Au besoin, l'utilisation d'une architecture de ce type permetterait d'ajouter sans enjeux réels d'autres services pour les besoins du projet.

1.2 Communication

En ce qui concerne l'envoie de données entre les services, l'architecture prévoie utiliser deux types de communication IPC

POSIX Socket Envoie des commandes entre les services.

Shared Memory Region Partage des données télémétriques et commandes ayant une contrainte de temps (*Real Time Processing*)

1.3 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher

2 Système Linux embarqué

Date de décision : 15 octobre 2026
Proposé par : Marc-Antoine
Statut : Adopté à l'unanimité
Absences : Aucune

2.1 Présentation

Afin d'accélérer la vitesse d'acquision et apprendre d'avantage sur l'utilisation de Linux en système embarqué, La proposition suivante utiliserait un système minimal *Buildroot* pour pouvoir compiler et déployer une distribution Linux.

2.2 PID 1

Contrairement à *Raspberry PI OS*, le système d'exploitation aurait un programme d'entré minimal sous forme de service. Celui-ci serait responsable d'attacher les bibliothèques et les pilotes nécessaires pour les sous services approuvé au point 1. De plus, Ce service doit démarrer les services et s'assurer de redémarrer ceux ayant terminés. En aucun cas, le **ServiceManager** doit sortir de la boucle. Autrement, le kernel Linux lancera une erreur et obligera un redémarrage du système.

2.3 Paquets ajoutés

wpa_supplicant Permettera d'utiliser le module WIFI et intéragir avec le pilote

libgpiod2 Communiquera avec le pilote responsable des *GPIO*

i2c-dev Intéragira avec les contrôlleurs I^2C pour les moteurs et le capteur de ligne.

sh Exécution des scripts *shell* dans le terminal

dropbear Établiera une communication SSH avec l'utilisation du port Ethernet statique ou du WIFI

2.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher

3 Avancements et modification

Date de décision : 2 novembre 2026
Proposé par : Jonathan Valade
Statut : Adopté à l'unanimité
Absences : Aucune

3.1 Avancements

Au cours de la journée, l'architecture embarquée a majoritairement été commencé pour retrouver les services principaux. L'utilisation d'un seul *Shared memory region* pour représenter le robot pour tout les services à été choisi pour des questions de simplicités. Le système Ethernet permettait d'entrer sur le robot de manière fiable et efficace. En utilisant un utilitaire *SCP* pour envoyer les services à des fins de déverminage, l'exécution de *scripts* permettant d'automatiser le deployement de l'environement de test permettait de rapidement avoir un prototype testable. Il a été possible d'évaluer des constantes pour calibrer le capteur de ligne afin que le robot puisse normaliser et moyenner, par le biais d'algorithmes, une ligne.

3.2 Modification

Afin de démarrer le robot et le WIFI sans à avoir à utiliser le cable ethernet, la proposition décrit l'utilisation de deux boutons et l'intégration d'un service **PeripheralsService** pour détecter l'intéraction avec les boutons et exécuter l'action suivant lesdites intéractions. De plus, un système sonore permetterais de confirmer que le robot est démarré ou bien de connaître l'état des systèmes (WIFI ouvert/fermé, démarrage de la séquence, etc.)

3.3 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher

4 Formules GoDot

Date de décision : 2 novembre 2026
Proposé par : Jérémie Perron
Statut : Adopté à l'unanimité
Absences : Aucune

4.1 Présentation

Le rapport d'équations *Filtre tes connaissances* utilisé pour la simulation GoDot est disponible. Les équations présentées dans le rapport seront validées par simulation avant d'être implémentées dans le robot.

4.2 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher

5 Avancements et modification

Date de décision : 3 novembre 2026
Proposé par : Tristan Faucher
Statut : Adopté à l'unanimité
Absences : Aucune

5.1 Avancements

Le développement d'aujourd'hui a majoritairement de finaliser les services de base et de commencer la télémétrie. Des outils à utilisation secondaire au projet ont aussi été réalisé avec l'assistance d'intelligence artificielle pour alégé le temps de développement. Le service responsable des décisions devrait fonctionner au plus tard demain lorsque les simulations GoDot seront terminé.

5.2 Simulation GoDot (Raphaël Bonhomme)

La simulation présentait quelques imperfection vis-à-vis l'accélération dans les courbes. De plus, le robot simulé allait beaucoup trop droit comparativement à celui utilisé dans le projet. Une solution simple a été d'implémenter des imperfections dans la simulation afin que le modèle subisse des forces extérieurs pour nuire au mouvement.

5.3 Modification

Pour optimiser le déploiement des constantes entre la simulation et GoDot, la proposition suivante recommande d'utiliser un service tier qui connectera GoDot et les services embarqués. L'objectif est de placer les services dans un container *Docker* afin de connecter localement GoDot aux services.

5.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher

6 Avancements

Date de décision : 4 novembre 2026
Proposé par : Tristan Faucher
Statut : Adopté à l'unanimité
Absences : Aucune

6.1 Capteurs de distance (Jonathan Valade)

Le service responsable des capteurs possède maintenant la capacité de détecter des objets en utilisant le capteur ultrason. Celui-ci utilise les broches *GPIO* pour envoyer une ondes et attendre la réception.

6.2 Shared memory Region (Marc-Antoine Gauthier)

Au lieu d'utiliser une seule région partagée entre les services, le système utilise maintenant une région par service pour le transfert sans délai d'information. Cela allège les changements de contextes et isole correctement les fichiers mémoires des services.

6.3 Fichiers de configuration

Afin de ne pas à avoir à recompiler les services lors d'un changement de paramètre, le système prend en charge l'utilisation de fichiers pour configurer le service. Modifiable par le service de *Télémétrie* via des **POSIX sockets**, cette amélioration simplifiera les modifications externes par le biais d'un utilitaire externe.

6.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher